

CLOUD COMPUTING

Complete Study Notes — Day 1 & Day 2

Topics Covered

Chapter 1 — What is Cloud?	Chapter 2 — Public vs Private Cloud
Chapter 3 — Virtualization	Chapter 4 — AWS vs Others
Chapter 5 — IAM in AWS	Chapter 6 — Practical: S3 + IAM Lab
Chapter 7 — EC2 in AWS	Chapter 8 — Practical: Jenkins on EC2



CHAPTER 7 — What is EC2 & Why EC2?



Simple Explanation — Rent vs Buy

Old Way — Buy Physical Server

Wait 2-3 months for server delivery
Set up data center — huge cost
Cost = Crores of rupees upfront
Low traffic = idle servers = wasted money
Traffic spike = server crashes

EC2 — Rent Virtual Server on AWS

Ready in 2 minutes — no waiting
AWS manages all hardware for you
Pay per hour/second — zero upfront
Low traffic = shut down = stop paying
Traffic spike = auto scale instantly



Technical Definition



Definition:

EC2 (Elastic Compute Cloud) is AWS's core compute service that allows you to rent virtual servers (called Instances) in the cloud. You choose the OS, processing power, memory, and storage — server is ready in minutes.

Term	Meaning
Elastic	Can grow or shrink based on your needs at any time
Compute	Processing power — CPU, RAM, running your applications
Instance	One virtual server running on EC2
AMI	Amazon Machine Image — the OS blueprint to launch an instance
Key Pair (.pem)	Security file used to SSH login to your Linux server
Security Group	Firewall that controls which traffic can reach your instance

Real Life Analogy — Apartment vs House

Buying a Physical Server = Buying a house
Huge upfront cost. You maintain everything. Stuck with it forever.

EC2 Instance = Renting a fully furnished apartment
Pay only for days you stay. Everything is already set up.
Move out (terminate) anytime. Upgrade to bigger apartment in minutes.

EC2 Instance Types — Choose Your Server Size

Instance Family	Best For	Real Example
t3 / t4g (General)	Small websites, testing, learning	Your first web app — Free tier eligible
c6i / c7g (Compute)	Heavy processing, gaming servers	Video encoding, batch jobs
r6i / x2i (Memory)	Big databases, data analytics	Oracle DB, SAP workloads
p4 / g5 (GPU)	AI, Machine Learning, 3D rendering	Training ML models
i3 / d3 (Storage)	Massive data storage needs	Data warehousing

4 EC2 Pricing Models

Pricing Model	How It Works + Best For
On-Demand	Pay per hour/second. No commitment. Best for: testing, unpredictable workloads
Reserved	Commit 1 or 3 years. Save up to 72%. Best for: production servers 24/7
Spot (Cheapest)	Use AWS unused capacity. Save up to 90%. Can be terminated 2 min notice. Best for: batch jobs, ML
Dedicated Host	Entire physical server only for you. Most expensive. Best for: banks, govt compliance

EC2 Security — Key Concepts

Concept	Explanation
Security Group	A firewall around your EC2. Controls which traffic can IN and OUT

Inbound Rule — Port 80	Allow HTTP from anywhere so your website is accessible
Inbound Rule — Port 22	Allow SSH only from your IP so only you can login
Inbound Rule — Port 8080	Jenkins default port — must be opened to access Jenkins on browser
Key Pair (.pem file)	Your SSH login key to connect to the server. NEVER lose or share it

🔗 **EC2 = A virtual computer on AWS that you can rent in minutes, scale in seconds, and pay only for what you use — without ever touching physical hardware.**

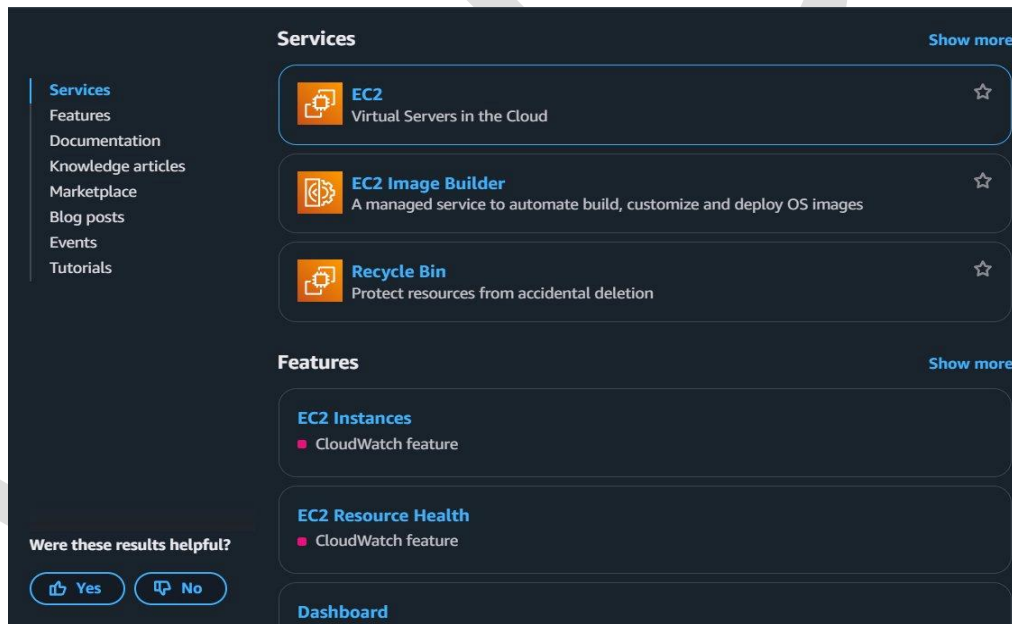


CHAPTER 8 — Practical: Deploy Jenkins on EC2

Launched an EC2 Instance (Ubuntu, t3.micro) on AWS
Created a Key Pair and SSH'd into the server from Windows Terminal
Installed Java (required for Jenkins) on the EC2 instance
Installed and started Jenkins on the EC2 instance
Edited Security Group to open Port 8080 for Jenkins
Accessed Jenkins on the browser via Public IP
Unlocked Jenkins and reached the setup screen

PART 1 — Launch EC2 Instance

Step 1 — Search EC2 in AWS Console



Search EC2 in AWS Console — Select EC2: Virtual Servers in the Cloud

Step 2 — Open Instances and click Launch Instances



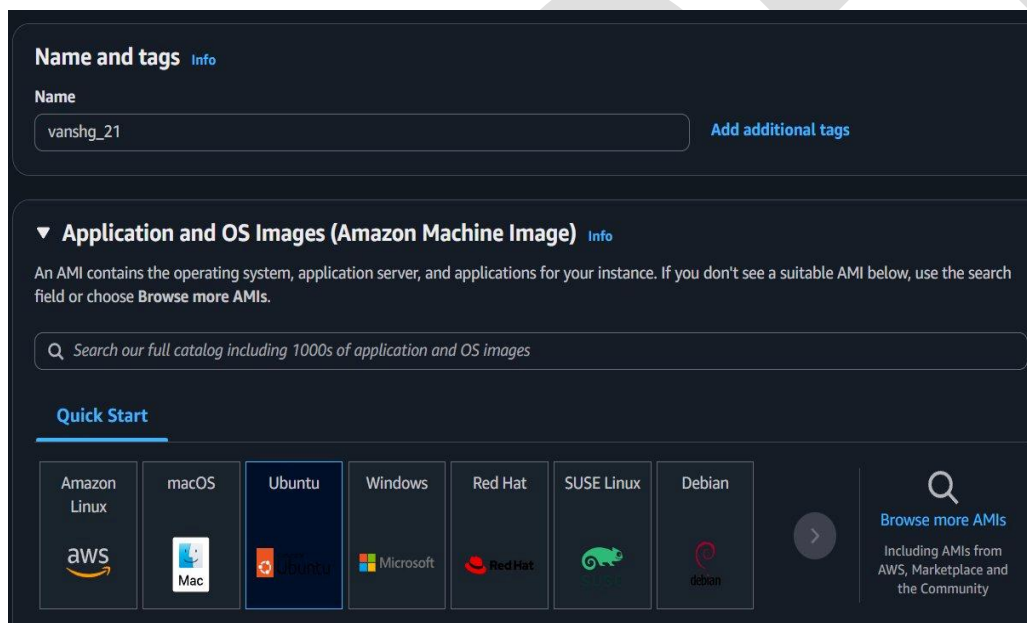
EC2 Instances Dashboard — No instances running yet. Click orange Launch Instances button.

Step 3 — Name your instance and select Ubuntu as the AMI

Instance Name: vanshg_21

AMI Selected: Ubuntu (Free tier eligible)

AMI = Amazon Machine Image — the OS blueprint used to create your server



Name the instance 'vanshg_21' and select Ubuntu as the AMI

Step 4 — Select Instance Type t3.micro and create Key Pair

Instance Type: t3.micro — Free tier eligible (2 vCPU, 1 GiB Memory)

Key Pair Name: vanshg_21

IMPORTANT: Download and save the .pem file immediately. You cannot download it again!

The screenshot shows the AWS Management Console interface for launching an instance. The 'Instance type' section is expanded, showing 't3.micro' as the selected instance type, which is 'Free tier eligible'. It lists specifications: Family: t3, 2 vCPU, 1 GiB Memory, Current generation: true. Pricing information is also displayed for various operating systems. To the right, there is a toggle for 'All generations' and a link to 'Compare instance types'. Below this, the 'Key pair (login)' section is expanded, showing a text box with 'vanshg_21' and a 'Create new key pair' button.

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t3.micro Free tier eligible

Family: t3 2 vCPU 1 GiB Memory Current generation: true

On-Demand Linux base pricing: 0.0112 USD per Hour On-Demand SUSE base pricing: 0.0112 USD per Hour

On-Demand Windows base pricing: 0.0204 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.0147 USD per Hour On-Demand RHEL base pricing: 0.04 USD per Hour

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

vanshg_21 [Create new key pair](#)

Instance type t3.micro selected — Key Pair vanshg_21 created

Step 5 — Launch the instance — Success!

The screenshot shows the 'Launch an instance' page in the AWS Management Console. A green banner at the top indicates 'Success: Successfully initiated launch of instance (i-0afb732967c63506a)'. Below this, there is a 'Launch log' section and a 'Next Steps' section with a search bar. The 'Next Steps' section contains four cards: 'Create billing usage alerts', 'Connect to your instance', 'Connect an RDS database', and 'Create EBS snapshot policy'. Each card has a description and a button to perform the action.

EC2 > Instances > Launch an instance

Success
Successfully initiated launch of instance (i-0afb732967c63506a)

► Launch log

Next Steps

What would you like to do next with this instance, for example "create alarm" or "create backup"

Create billing usage alerts
To manage costs and avoid surprise bills, set up email notifications for billing usage thresholds.
[Create billing alerts](#)

Connect to your instance
Once your instance is running, log into it from your local computer.
[Connect to instance](#)
[Learn more](#)

Connect an RDS database
Configure the connection between an EC2 instance and a database to allow traffic flow between them.
[Connect an RDS database](#)
[Create a new RDS database](#)
[Learn more](#)

Create EBS snapshot policy
Create a policy that automates the creation, retention, and deletion of EBS snapshots
[Create EBS snapshot policy](#)

Instance launched successfully — Instance ID: i-0afb732967c63506a

🔗 PART 2 — Connect to EC2 via SSH

Step 6 — Copy the Public IPv4 address from Instance Summary

Public IPv4 Address: 35.154.133.59

Instance State: Running

Instance Type: t3.micro | Region: ap-south-1 (Mumbai)

Instance summary for i-0afb732967c63506a

Updated less than a minute ago

Instance ID: i-0afb732967c63506a

IPv6 address: -

Hostname type: IP name: ip-172-31-43-184.ap-south-1.compute.internal

Answer private resource DNS name: IPv4 (A)

Auto-assigned IP address: 35.154.133.59 [Public IP]

Public IPv4 address: 35.154.133.59 | open address

Instance state: Running

Private IP DNS name (IPv4 only): ip-172-31-43-184.ap-south-1.compute.internal

Instance type: t3.micro

VPC ID: vpc-053578668b4272bc7

Private IPv4 addresses: 172.31.43.184

Public DNS: ec2-35-154-133-59.ap-south-1.compute.amazonaws.com | open address

Elastic IP addresses: -

AWS Compute Optimizer finding: Opt-in to AWS Compute Optimizer for recommendation s. | Learn more

Instance Summary — Public IPv4: 35.154.133.59 | State: Running

Step 7 — SSH into the EC2 Instance from Windows Terminal

```
ssh -i vanshg_21.pem ubuntu@35.154.133.59
```

```
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\WELCOME\Downloads> ssh -i vanshg_21.pem ubuntu@35.154.133.59
The authenticity of host '35.154.133.59 (35.154.133.59)' can't be established.
ED25519 key fingerprint is SHA256:MzvAJiwdhAVIQvMnCqWQcupxJRbN0sT5vjwa5aalCVA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '35.154.133.59' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1004-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri May 29 09:12:03 UTC 2026

  System load:  0.0          Temperature:    -273.1 C
  Usage of /:   30.4% of 6.61GB Processes:      116
  Memory usage: 26%         Users logged in: 0
  Swap usage:   0%          IPv4 address for ens5: 172.31.43.184

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

Successfully connected — Ubuntu 26.04 LTS running on AWS EC2 Mumbai

Connection confirmed — Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1004-aws x86_64)
Memory usage: 26% | Private IPv4: 172.31.43.184
You are now INSIDE your cloud server running in AWS Mumbai region!

⚙️ PART 3 — Install Java and Jenkins

Jenkins is built on Java — it cannot run without Java installed first.

```
# Step 1 — Install Java (required for Jenkins)
sudo apt install openjdk-11-jdk

# Step 2 — Add Jenkins GPG key
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian/jenkins.io-2026.key

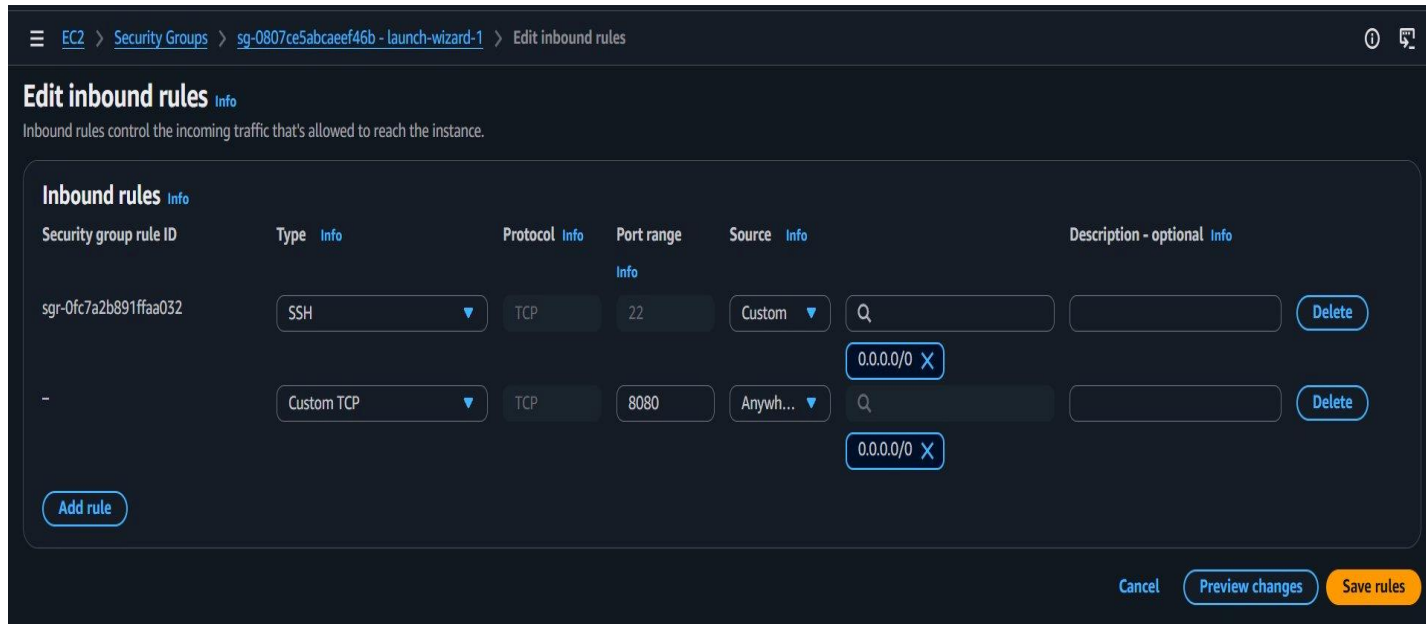
# Step 3 — Add Jenkins to apt sources list
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
  https://pkg.jenkins.io/debian binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

# Step 4 — Update package list and install Jenkins
sudo apt update
sudo apt install jenkins

# Step 5 — Verify Jenkins is running
sudo systemctl status jenkins
```

PART 4 — Open Port 8080 in Security Group

Jenkins runs on Port 8080 by default. AWS blocks all ports by default — you must manually open Port 8080 in the EC2 Security Group inbound rules.



EC2 > Security Groups > sg-0807ce5abcaef46b - launch-wizard-1 > Edit inbound rules

Edit inbound rules Info

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules Info

Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
sgr-0fc7a2b891ffaa032	SSH	TCP	22	Custom	Q	Delete
-	Custom TCP	TCP	8080	Anywh...	Q	Delete

0.0.0.0/0 X

0.0.0.0/0 X

Add rule

Cancel Preview changes Save rules

Security Group — SSH (Port 22) kept + Port 8080 opened for Jenkins (0.0.0.0/0)

Rule Added	Purpose
Port 22 — SSH — Custom (your IP)	Allows you to SSH into the server securely
Port 8080 — Custom TCP — Anywhere	Allows browser access to Jenkins from any IP

PART 5 — Access Jenkins on Browser & Complete Setup

Open your browser and navigate to:

```
http://35.154.133.59:8080
```

Step — Unlock Jenkins with initial admin password

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
/var/lib/jenkins/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue

Unlock Jenkins — Password stored at /var/lib/jenkins/secrets/initialAdminPassword

```
# Get Jenkins initial admin password
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Step — Select Install Suggested Plugins

Getting Started

Customize Jenkins

Plugins extend Jenkins with additional features to support many different needs.

Install suggested plugins

Install plugins the Jenkins community finds most useful.

Select plugins to install

Select and install plugins most suitable for your needs.

Jenkins 2.566

Customize Jenkins — Click Install suggested plugins to complete setup

Complete Flow — What Happened Today

1. Searched EC2 on AWS Console and opened the service
↓
2. Launched EC2 — Ubuntu, t3.micro, Key Pair: vanshg_21
↓
3. Got Public IP: 35.154.133.59 — Instance state: Running
↓
4. SSH connected: `ssh -i vanshg_21.pem ubuntu@35.154.133.59`
↓
5. Installed Java: `sudo apt install openjdk-11-jdk`
↓
6. Added Jenkins repo, ran apt update, installed Jenkins
↓
7. Opened Security Group — added Port 8080 inbound rule
↓
8. Browser: `http://35.154.133.59:8080` — Jenkins loaded!
↓
9. Retrieved admin password from server and unlocked Jenkins
↓
10. Installed suggested plugins — Jenkins is fully deployed!

Key Learnings from This Practical

Concept	What the Practical Proved
EC2 is a real server	You SSH'd into it — behaves exactly like a Linux machine
AMI matters	Ubuntu chosen because Jenkins runs best on Linux
Key Pair is critical	Without <code>vanshg_21.pem</code> you cannot login to the server
Security Groups are gatekeepers	Port 8080 had to be manually opened — nothing opens by default
Jenkins needs Java	Software dependencies are real — Java must come before Jenkins
Public IP is your address	<code>35.154.133.59:8080</code> is how the world reaches your Jenkins

Day 2 Complete — You launched a real cloud server, connected to it via SSH, installed software, configured security, and deployed Jenkins. This is real DevOps work!

VANISH